Product Requirements & Prioritization

Multi-Tenant SaaS API — approved scope, functional requirements, non-functional requirements and priorities

1. Product Definition

Product: Multi-Tenant SaaS API

Primary purpose: a production-grade, organization-based project and task management API. The immediate objective is an engineering-first learning/portfolio project with architectural foundations that can support commercialization later.

2. Scope and Assumptions Confirmed

- Target is initially a learning project; future commercialization is possible.
- Multi-tenancy is organization-based for the first version.
- Users can belong to organizations; more advanced multiple-membership models are future scope.
- Initial roles: Organization Owner, Organization Admin, Project Manager, Member, Viewer.
- Permissions exist at Organization and Project level.
- Tasks are full-featured: title, description, status, priority, assignee, due date, labels, project, creator, comments and future file metadata.
- Actual file uploads are deferred; only file metadata is planned for the current scope.
- Web frontend is the primary API consumer.
- Authentication starts with JWT plus refresh tokens; 2FA/OAuth and similar mechanisms are future scope.
- Initial scale target is 100K users.
- Redis is used for rate limiting, caching, session/token management and distributed locks.
- Kafka is preferred for background/event-driven processing; Redis-backed jobs are the fallback if needed.
- Observability includes structured logging, Prometheus, Grafana, OpenTelemetry/tracing, request correlation and health/readiness checks.
- Testing includes unit, integration, API/E2E, Testcontainers and later load/security testing.

3. Functional Requirements

ID	Requirement	Description	Priority
AUTH-001	User registration	A user can register using email/password.	P0
AUTH-002	Login	A registered user can authenticate and receive an access token.	P0
AUTH-003	JWT access token	Protected API requests can be authenticated using JWT access tokens.	P0
AUTH-004	Refresh token	A valid refresh token can obtain a new access token.	P0
AUTH-005	Session management	Sessions/refresh tokens can be tracked and revoked.	P0
AUTH-006	Logout	A user can invalidate the active session.	P0
AUTH-007	Password reset	A user can request and complete password reset.	P1

ID	Requirement	Description	Priority
AUTH-008	Email verification	A user can verify their email address.	P1
ORG-001	Organization creation	An authenticated user can create an organization.	P0
ORG-002	Membership	Users can belong to organizations.	P0
ORG-003	Member management	Authorized users can manage organization members.	P0
ORG-004	Tenant isolation	Users cannot access another organization's resources without authorization.	P0
RBAC-001	Organization RBAC	Organization permissions are enforced for protected actions.	P0
RBAC-002	Project RBAC	Project permissions are enforced for project resources.	P0
PROJ-001	Project CRUD	Authorized users can create, read, update and archive/delete projects.	P0
PROJ-002	Project membership	Projects can have members and project-level permissions.	P0
TASK-001	Task management	Authorized users can create, read, update and delete/archive tasks.	P0
TASK-002	Task attributes	Tasks support status, priority, assignee, due date and labels.	P0
COMMENT-001	Comments	Authorized users can create, read, update and delete comments.	P0
FILE-001	File metadata	The system can store file metadata without implementing file upload/storage.	P3
QUERY-001	Pagination	Collection APIs support pagination.	P1
QUERY-002	Filtering	Collection APIs support relevant field filtering.	P1
QUERY-003	Sorting	Collection APIs support controlled sorting.	P1
QUERY-004	Search	Users can search relevant projects/tasks/users using PostgreSQL initially.	P1
NOTIF-001	Email notifications	Defined domain events can produce asynchronous email notifications.	P1
INFRA-001	Rate limiting	The API enforces configurable rate limits using Redis.	P1
INFRA-002	Caching	Selected read paths can use Redis caching where useful.	P1
INFRA-003	Distributed locks	Critical operations can use distributed locks where justified.	P1
INFRA-004	Background processing	Long-running/non-request work is processed asynchronously.	P1
OBS-001	Structured logging	Application logs are structured and correlation-friendly.	P1
OBS-002	Metrics	Prometheus metrics are exposed for important service behavior.	P1
OBS-003	Dashboards	Grafana dashboards visualize operational metrics.	P1
OBS-004	Tracing	OpenTelemetry-based distributed tracing is supported.	P1
OPS-001	Graceful shutdown	The service drains requests and closes dependencies cleanly.	P0
OPS-002	Health/readiness	Health and readiness endpoints are available.	P1
API-001	API versioning	The API supports explicit versioning, initially v1.	P0
API-002	OpenAPI	The API is documented with OpenAPI/Swagger.	P0
TEST-001	Unit tests	Business logic has unit test coverage.	P1
TEST-002	Integration tests	Database/Redis/Kafka integrations have automated tests.	P1
TEST-003	API/E2E tests	Important end-to-end workflows are tested.	P2
TEST-004	Load/security testing	Performance and security testing are performed before production hardening.	P2
DEVOPS-001	Docker	The application and local dependencies can run through containers.	P2
DEVOPS-002	CI/CD	Build, test and deployment workflows are automated.	P2

4. Non-Functional Requirements

ID	Area	Requirement
NFR-001	Scale	Architecture should be capable of evolving toward approximately 100K users.
NFR-002	Security	Tenant isolation, authentication, authorization, password hashing, token controls and safe error handling are mandatory.
NFR-003	Performance	Avoid unnecessary synchronous work in request paths; use caching and asynchronous processing where justified.
NFR-004	Reliability	Support graceful shutdown, timeouts, retries where appropriate, health/readiness checks and dependency failure handling.
NFR-005	Observability	Logs, metrics, traces and operational dashboards should make failures and latency diagnosable.
NFR-006	Maintainability	Use modular boundaries and consistent patterns rather than a feature-by-feature collection of ad-hoc code.
NFR-007	Testability	Core business logic and infrastructure integrations must be testable in isolation and together.
NFR-008	API consistency	Use consistent validation, errors, pagination, filtering, sorting and versioning conventions.

5. Prioritization

Priority	Meaning	Scope
P0	MVP / blocker	Foundation, authentication, organization/membership, RBAC, projects, tasks, comments, tenant isolation, API versioning, OpenAPI, migrations, middleware, request context, error handling and graceful shutdown.
P1	Important for first production-like release	Pagination, filtering, sorting, search, Redis capabilities, Kafka/background processing, email notifications, password reset, email verification, logging, metrics, dashboards, tracing and health/readiness.
P2	Production hardening	E2E tests, load testing, security testing, Docker, CI/CD, deployment automation, advanced failure/retry patterns and performance optimization.
P3	Future commercial product	Actual file uploads/object storage, 2FA, OAuth, SSO, billing, subscriptions, usage limits, audit logs, multiple membership models, advanced analytics, real-time collaboration and advanced search infrastructure.

6. Definition of Done

A feature is not considered complete merely because its endpoint works. A completed feature should normally include schema/migrations, validation, authorization, tenant isolation, business rules, consistent errors, observability, documentation and appropriate automated tests. Higher-priority production features should also include failure handling and operational visibility.